

Chunking in Retrieval-Augmented Generation

Retrieval-augmented generation answers a question in two steps. First it finds passages that look relevant, then it asks a language model to answer from those passages alone. The passages are chunks: pieces of the original documents, cut before anything is indexed. How they are cut decides what the retriever can ever find.

Why chunk boundaries matter

A retriever scores each chunk as a whole. When a boundary falls in the middle of an explanation, the question lands on one half and the answer on the other, and neither half scores well on its own. The model then receives a passage that mentions the topic but never resolves it.

Boundaries also decide what travels together. A definition separated from the term it defines, or a table separated from its caption, loses the context that made it useful. The embedding of such a chunk describes a fragment, not an idea, so it drifts away from the questions it should match.

A useful rule of thumb: a chunk should answer one question well. If you cannot say in a sentence what a chunk is about, a retriever will struggle to decide when it is relevant, and a reader will struggle to use it.

Bad boundaries are hard to see from the final answer. The model is fluent either way, so the symptom is usually an answer that is vague or quietly wrong. Looking at the chunks themselves, before indexing, is the quickest way to catch the problem.

Chunk size and overlap

Chunk size is a trade between precision and context. Small chunks match a question closely, because every sentence in them is about the same thing. Large chunks carry more surrounding context, but their embedding averages over several topics and matches none of them sharply.

There is also a budget on the other side. Every retrieved chunk is pasted into the prompt, so the number of chunks you retrieve times their size must fit in the context window, with room left for the question and the answer. Doubling the chunk size roughly halves how many distinct passages the model can see.

Overlap repeats the end of one chunk at the start of the next. It is a cheap insurance policy against a boundary that lands in the wrong place: a sentence cut at the edge of one chunk appears whole in its neighbour. The cost is a larger index and near-duplicate results, since adjacent chunks now share text.

A common starting point is a few hundred tokens per chunk with an overlap of ten to twenty percent. Treat these numbers as a first guess to measure, not a rule. Documents with short, self-contained entries, such as a glossary or an FAQ, often want much smaller chunks than long narrative reports.

Overlap should always be smaller than the chunk itself. When the two are equal, each step moves forward by nothing, and a splitter either loops or refuses to run.

Structure-aware splitting

Fixed-size splitting ignores the document's own shape. Structure-aware splitting uses it: headings, paragraphs, list items and tables become the natural places to cut. A section under one heading usually discusses one subject, which is exactly what a good chunk should do.

This only works when the parser recovers the structure. A layout-aware parser labels headings, footers and tables, while a plain text extractor returns a flat stream of lines. Cleaning comes next: running headers, footers and page numbers are removed, so they do not end up in the middle of chunks.

A useful rule of thumb: a chunk should answer one question well. If you cannot say in a sentence what a chunk is about, a retriever will struggle to decide when it is relevant, and a reader will struggle to use it.

A structure-aware splitter still needs a size limit. A long section is split again inside itself, preferably at paragraph and then sentence boundaries, and each piece keeps its heading path so that the retriever knows where it came from.

Choosing a starting point

Start simple, look at the chunks, then ask a handful of real questions and check which chunks come back. Change one setting at a time and compare the results side by side. The right configuration is the one that retrieves the passage a careful reader would have picked.